

Photometric Pipeline Participant Handbook

Everything you need for the session: install the pipeline on your own machine, put a night of observatory data where it belongs, and work through five notebooks that take raw frames to a calibrated magnitude — with an error bar you can defend.

YOU WILL PRODUCE	NOTEBOOKS	DISK NEEDED	PYTHON
one star's mag $\pm$ error	5, run in order	~1 GB	3.10 or newer

Contents

1	Before you start	2
2	Install	2
3	Where the data goes	4
4	Running the notebooks	5
5	Notebook by notebook	6
6	What to hand in	8
7	When something breaks	8

1 Before you start

Do the install **before** the session, not during it. Building the environment downloads a few hundred megabytes and takes ten to twenty minutes on a normal connection; doing it in the room costs everyone the first hour.

WHAT	WHY
A laptop running Linux or macOS	One install route for both, on Intel and ARM alike. Windows works too, but through WSL — so start that part early, it needs a reboot.
~1 GB free disk	267 MB for the night's raw frames, ~325 MB for what the pipeline writes, plus the environment. On Windows, allow another ~2 GB for WSL.
Internet, at least once	Installing, and Phase 3's reference-catalog query. Both cache, so a second pass runs offline.
Comfort with a terminal	You will type about six commands. No prior astronomy software needed.

BRING THIS

Your assigned star — the instructor gives you either a catalog NUMBER or an RA/Dec pair. Measuring it is the session's deliverable.

2 Install

One route, on every operating system: get **Miniforge**, then run one script. Windows gets there through WSL — see below, and do that part first.

WHY MINIFORGE AND NOT PLAIN PIP

Phase 2 plate-solves your images against a star catalog, and every solver that can do it is a compiled binary rather than a Python package. Miniforge is what supplies one, along with prebuilt versions of the whole scientific stack, so nothing has to compile on your laptop.

Which platform you are on

YOUR MACHINE	ROUTE	NOTES
Linux, x86-64	<code>./install.sh</code>	The reference platform.
Linux, ARM (aarch64)	<code>./install.sh --conda</code>	Works, but only with ASTAP — nothing else is published for ARM Linux.
macOS, Intel	<code>./install.sh</code>	Identical to Linux.
macOS, Apple Silicon	<code>./install.sh</code>	Identical. See the Gatekeeper note below.
Windows	WSL, then the Linux route	Do the WSL part first; it needs a reboot.

Windows only: get to Linux first

The pipeline is developed and tested on Linux and macOS, and `install.sh` is a shell script. WSL gives you real x86-64 Linux on your Windows machine, so every instruction in this handbook then applies unchanged — and it is the route that is actually tested, which is why it is the one documented.

From an **administrator** PowerShell:

```
wsl --install
```

Reboot when it asks, open **Ubuntu** from the Start menu, and let it finish creating your user account. Everything from here on happens inside that Ubuntu window, and every command is the Linux one.

KEEP YOUR FILES ON THE LINUX SIDE

Your Windows drives appear under `/mnt/c`, so data you downloaded in Windows is reachable. But keep the repository and the work directory in your WSL home — reducing a night across `/mnt` is several times slower.

RUNNING `INSTALL.PS1` DOES NOTHING

It installs nothing on purpose. It prints these WSL steps and stops, so a Windows participant is not left with an environment that builds and then cannot solve. `cassa-doctor` reports native Windows the same way.

Step 1 — Install Miniforge

Miniforge is a minimal conda that defaults to the conda-forge channel. If you already have conda or mamba, skip this step.

The installer name is built from your own machine, so this is the same pair of commands on Linux, on macOS (Intel and Apple Silicon alike), and inside WSL:

```
curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh
```

Accept the defaults, then **close and reopen your terminal** so conda is on your PATH. Check it:

```
conda --version
```

On macOS you also need Apple's command-line tools, once, for git:

```
xcode-select --install
```

Step 2 — Clone and install

Identical everywhere:

```
git clone https://github.com/cassaiub/observatory.git
cd observatory/photometric_pipeline
./install.sh
```

The script builds a conda environment called `cassa-photometry` from `environment.yml`, installs the pipeline into it, adds whichever plate solver your platform can run, and finishes by checking

its own work. Expect ten to twenty minutes and a few hundred megabytes.

On ARM Linux add `--conda`: the in-process solver publishes no ARM wheel, so the conda route plus ASTAP is the combination that works there.

Step 3 — Activate and verify

```
conda activate cassa-photometry
cassa-doctor
```

`cassa-doctor` is the one command that proves the install: it reports your platform, the package version, the environment, which plate solver you have and which one a run would use, and where the sky data will come from. **Run it before the session, and bring the output if anything is not green.** You need to run `conda activate cassa-photometry` in every new terminal.

WHICH SOLVER YOU GET, AND WHY IT DOES NOT MATTER

The installer tries three, in order, and the first that installs wins: **ASTAP** (a sub-megabyte binary that works on every platform, including ARM), then Astrometry.net's `solve-field`, then an in-process Python solver. `cassa-doctor` tells you which one you have.

Your results do not depend on the answer. The one thing ASTAP does not report — the astrometric residual `ASTRMS`, which Phase 3 uses to size its catalog cross-match — the pipeline measures for itself, so every backend produces it. If your neighbour has a different solver, your catalogs are still comparable.

THERE IS NOTHING TO DOWNLOAD IN ADVANCE

Plate solving matches your stars against a sky catalog, and those catalogs are large — 859 MB for ASTAP, some 34 GB for Astrometry.net. You need neither. The pipeline reads your frame's pointing and field size, works out which pieces that field needs, and fetches only those: about **6 MB** for ASTAP or 165 MB for Astrometry.net, cached under `~/.cache/cassa-photometry/`. Pointing somewhere new later costs only the pieces the new field adds.

MACOS: GATEKEEPER MAY QUARANTINE ASTAP

If `cassa-doctor` finds the ASTAP binary but a solve fails with a security warning, macOS has quarantined the downloaded file. Clear it once: `xattr -d com.apple.quarantine "$(command -v astap_cli)"`

3 Where the data goes

The acquisition software writes a night as a tree, sorted by frame type and then by target. Copy that whole dated folder, unchanged, into `photometric_pipeline/workshop/raw/`:

```
workshop/raw/
|-- 20260903                                <- the night, as the software named it
    |-- BIAS/untargeted/                    15 frames
    |-- DARK/untargeted/                   15 frames
    |-- FLAT/untargeted/                   56 frames
    |-- LIGHT/m22/                         20 frames
        |-- 20260903T043707.171323_m22_Luminance_LIGHT_iub-dhaka....fits
```

Do not rename, flatten or sort anything. `RAW_DIR` is searched **recursively**, so the tree is read exactly as delivered — and a single flat directory of frames works too, if that is what you are given.

FOLDERS DO NOT DECIDE ANYTHING

Every frame is classified by its `IMAGETYP`, `FILTER` and `EXPTIME` headers, never by the folder it sits in. A frame filed under the wrong folder is still reduced as whatever it actually is — notebook 01 tells you when the two disagree.

Filters in this night

Blue Green Red Luminance

Five science frames in each, at 60 s. Flats exist for all four.

What the pipeline writes

Everything lands under `workshop/work/`, one directory per phase. Re-running a phase replaces its products in place rather than piling up copies.

DIRECTORY	WRITTEN BY	CONTENTS
<code>work/phase1</code>	notebook 02	<code>calibrated_*.fits</code> — SCI/ERR/DQ, in electrons
<code>work/phase2</code>	notebook 03	<code>Master_*.fits</code> — stacked and WCS-solved
<code>work/phase3</code>	notebook 04	<code>*_catalog.csv</code> , <code>*_fluxcal.fits</code> , segmentation maps

If your data lives somewhere else

Point at it instead of copying — the notebooks honour these environment variables:

```
export RAW_DIR=/path/to/the/night
export WORK_DIR=/path/for/outputs
```

4 Running the notebooks

Activate the environment, then start Jupyter from the notebooks directory:

```
conda activate cassa-photometry
cd observatory/photometric_pipeline/workshop/notebooks
jupyter lab
```

Run them in order. Notebook 00 checks your environment and 01 reads only the raw frames, but 02, 03 and 04 each consume the previous notebook's output. Within a notebook, run every cell from the top — later cells depend on names defined earlier.

The first cell of every notebook

Each notebook opens with the same config cell. It resolves `RAW_DIR`, `WORK_DIR` and the per-phase directories, then prints what it found — including your raw tree, directory by directory. If those paths look wrong, stop and fix them there; everything downstream inherits them.

Filling in the blanks

The exercises are partly written for you. Lines you must complete are marked:

```
gain = FILL_IN          # TODO 1: e-/ADU as Phase 1 resolved it
```

The surrounding code is scaffolding — leave it alone. Each blank takes one expression. If you run a cell without filling them you get `NameError: name 'FILL_IN' is not defined`, which names the exact line still waiting for you. Nothing fails silently.

Reading the answer tables

Every exercise heading carries a small table naming the one or two values it wants, with the expected answer beside each. Compare your numbers against it — a mismatch is worth raising in the room, because it usually means something real about your data rather than a mistake in your arithmetic.

5 Notebook by notebook

00 Setup ~5 min

Four checks that your machine can do the session at all. Run every cell.

What you are checking

- The package imports and reports a version.
- The astrometry index directory resolves and holds index files.
- `solve-field` is found — or is not, in which case the in-process solver takes over, which is fine.
- Your raw tree is found, and the frame counts per directory match what you copied in.

STOP IF the raw tree shows zero frames, or the index directory does not exist. Neither is recoverable later.

01 Raw frame health ~10 min

Are these frames worth reducing? The pipeline will happily reduce bad calibration frames and produce plausible numbers that are quietly meaningless — this is where that gets caught.

What it does

- Inventories every frame by type, filter and exposure, and flags any frame whose folder disagrees with its header.
- Measures each frame once — level, scatter, saturated fraction, temperature — and plots every frame's signal level against the ADC ceiling.
- Runs sixteen checks and prints a verdict.

The verdict

PROCEED Everything passed. Go on to 02.

CAUTION Nothing blocking, but the result will be weaker than it looks. Note the warnings — they explain later numbers.

STOP A blocking problem. Each one names a way the reduction will be *wrong* rather than merely noisy, and none can be repaired afterwards. Raise it with an instructor.

RECORD the verdict, and the headline of any FAIL.

02 Phase 1 — Calibration ~20 min

Bias, dark and flat come off; the frame is converted to electrons; and the error budget is seeded. Output is `calibrated_*.fits` carrying SCI, ERR and DQ.

The run itself takes several minutes for a full night — start it, then read the exercise text while it works.

Exercises

Ex 1 • 3 blanks **What did calibration actually change?** Find the instrumental pedestal that was removed, and the real sky left behind. Both in electrons.

Ex 2 • 2 blanks **Is the ERR plane just Poisson + read noise?** Find the ratio of actual to predicted error, and what the excess is.

RECORD four numbers: pedestal, sky, ratio, extra term.

03 Phase 2 — Integration & WCS ~20 min

Frames are aligned, stacked with inverse-variance weighting, and plate-solved against real stars. Output is `Master_*.fits` with a WCS.

Exercises

Ex 1 • 2 blanks **How much deeper is the stack?** Find the depth gain and the number of frames stacked. Compare against $\sqrt{N}$.

Ex 2 • 2 blanks **What did the plate solve buy us?** Find the plate scale in arcsec/pixel and the field of view in arcmin.

WATCH FOR a depth gain that falls short of $\sqrt{N}$. That is real, not an error: warping resamples pixels, which correlates neighbours, and correlated noise does not average down as fast as the formula assumes.

04 Phase 3 — Photometry & zero point ~25 min

Sources are detected and measured, a zero point is derived by cross-matching against a reference catalog, and a catalog with magnitudes and errors is written. This is the notebook that needs internet.

Exercises

Ex 1 • 2 blanks **Check the zero-point arithmetic.** Recompute magnitude and error from flux and the header; find the largest residual, and which term dominates for the brightest star.

Ex 2 • 2 blanks **Your assigned star.** Fill in `ASSIGNED_NUMBER` or `ASSIGNED_RADEC` at the top of the cell, then report the magnitude, its uncertainty, and which half of the error budget dominates.

IF THERE IS NO ZERO POINT

Phase 3 writes `_fluxcal.fits` only when it measures one. Without it the catalog still exists, carrying instrumental magnitudes, and the notebook falls back to those automatically — every shape you are asked about survives, only the zero of the scale is arbitrary. Exercise 1 skips itself and says so.

6 What to hand in

One line, from Exercise 2 of notebook 04:

ITEM	EXAMPLE
Which star	NUMBER 7, RA 278.98621, Dec -23.79178
Magnitude $\pm$ uncertainty	13.756 $\pm$ 0.017
Dominant error term	zero point
Filter and frames stacked	Blue, 3 frames

QUOTE IT PROPERLY

A magnitude without an uncertainty is not a measurement, and one written to more digits than its uncertainty supports is a claim you cannot defend. Round the value to the precision the error allows — the notebook does this for you, and it is worth understanding why.

7 When something breaks

WHAT YOU SEE	WHAT IT MEANS & WHAT TO DO
NameError: name 'FILL_IN' is not defined	A blank you have not filled yet. The traceback names the line.
NameError on a name from the config cell	Your kernel imported the config before it was last edited. Kernel → Restart , then run from the first cell.
Notebook 01 says STOP	A calibration frame is unusable. Do not reduce past it — the numbers will look fine and be wrong. Raise it in the room.
solver: solve-field not on PATH	Not a fault. Three backends can solve, and cassa-doctor's solver: in use line says which one you have. Only a problem if solver: any <i>fails</i> — that means none is installed.
no star database covering this field	ASTAP could not fetch the sky tiles for your pointing. Check your connection; the tiles are about 6 MB and are fetched during the solve.
macOS: ASTAP found, but the solve fails with a security warning	Gatekeeper has quarantined the downloaded binary. Clear it once with <code>xattr -d com.apple.quarantine "\$(command -v astap_cli)"</code>
Phase 3: <i>zero point failed</i> , no <code>_fluxcal.fits</code>	The cross-match found no usable stars in your image. The catalog is still written with instrumental magnitudes and the exercises adapt. Worth investigating, not worth blocking on.
Phase 3 hangs or times out	It is querying an online reference catalog. Check your connection; the query caches, so a second run is fast.

WHAT YOU SEE	WHAT IT MEANS & WHAT TO DO
conda not found	Miniforge is not installed, or you did not reopen the terminal after installing it. On Windows, make sure you are inside the Ubuntu (WSL) window, not PowerShell.
A phase takes far longer than the estimate	Normal on a laptop for a full night. Watch the progress bars; they move per frame. On WSL, also check the repository is in your Linux home rather than under /mnt/c — that alone can cost several times the runtime.

Still stuck? `cassa-doctor` prints the state of the whole install in one screen. Bring that output rather than a description.

CASSA Observatory • Independent University, Bangladesh — rooftop observatory, Dhaka.

Pipeline documentation lives in the repository: `README.md`, `docs/REFERENCE.md` and `docs/CUSTOMIZING.md`.